QC Core Insurance Walkthrough: From Insurance Card to Payment

A Visual Guide to the qc_core Database – The Garcia Family Story

Lenny Miller

April 2026

Contents

QC Core Insurance Walkthrough: From Insurance Card to Payment

A narrative guide to the `qc_core` database, telling the story of health insurance through the tables that make it work. Each "Act" follows a real-world event and maps it to the specific qc_core entities involved. For the formal DDD reference, see QC-Core-Domain-Analysis.

The Big Picture

Health insurance is a chain of events. A person gets coverage through their employer, picks a doctor, gets sick, sees the doctor, the doctor submits a bill, the insurance company decides how much to pay, and everyone gets notified. Every one of those steps lives in qc_core as tables and relationships.

Here's how the seven bounded contexts map to that chain:

```
REAL WORLD                          QC_CORE BOUNDED CONTEXT
---------------------------------   ---------------------------
Employer buys group plan         -> Benefit Administration
Employee enrolls, gets ID card   -> Member Services + Benefit Admin
Member picks a PCP               -> Provider Management
IPA codes auto-assigned weekly   -> Member Services (QCAP Sync)
Member needs specialist referral -> Referrals & Authorizations
Member visits doctor             -> Claims Processing (claim created)
Insurance adjudicates claim      -> Claims Processing (adjudication)
Provider gets paid               -> Payment Contracts + Accounting
Member gets EOB                  -> Accounting Services
```

Plexis High-Level Conceptual Diagram

This is the official Plexis relationship diagram showing how all the major components connect. Keep this in mind as we walk through each area in detail.

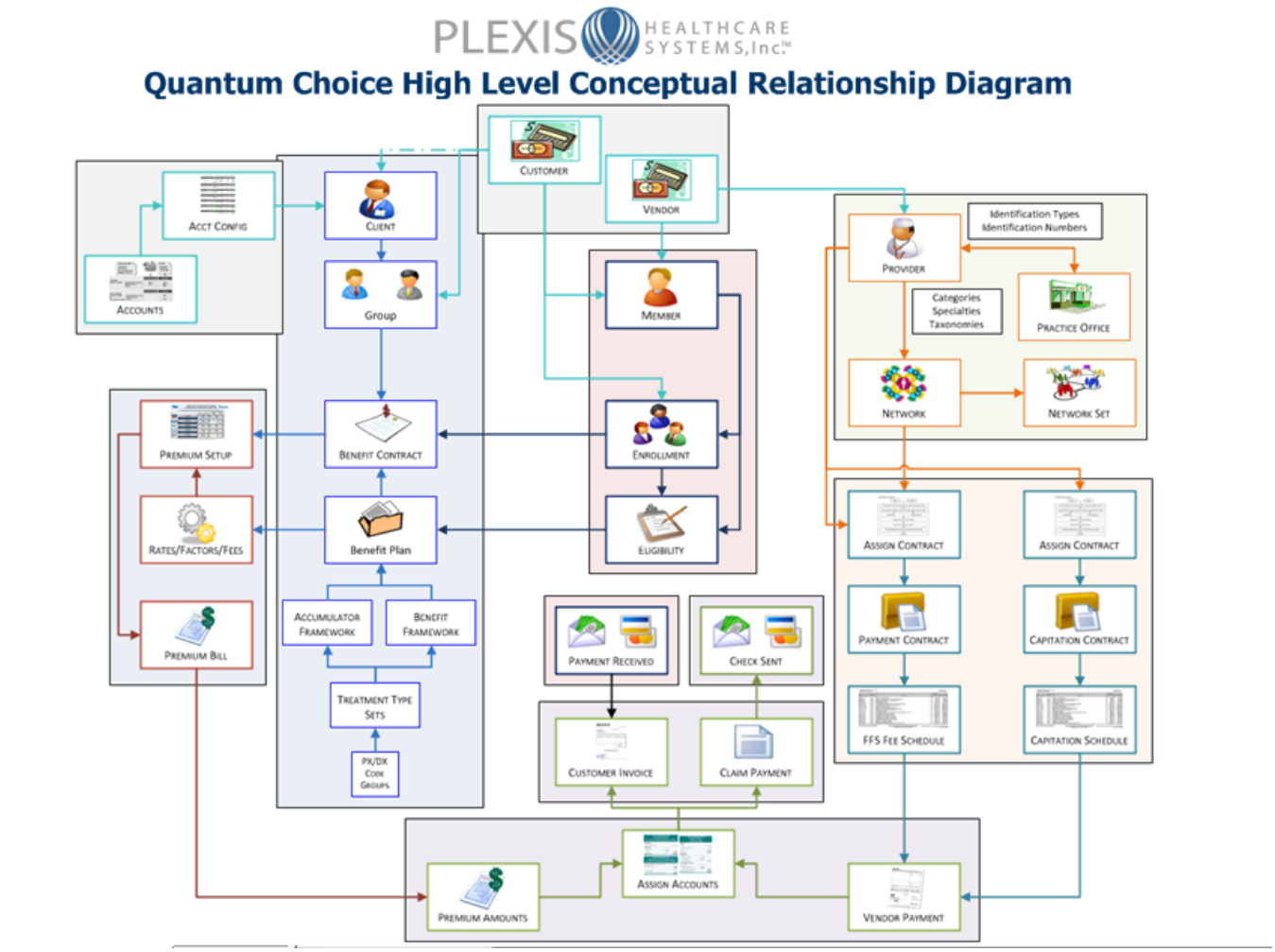

Figure 1: Plexis QC High-Level Conceptual Relationship Diagram

Master Entity Relationship Diagram

This shows the core entities across all contexts and how they connect. The `family_eligibility_member_benefit_plan` table is the central hub. Almost everything in qc_core eventually joins through it.

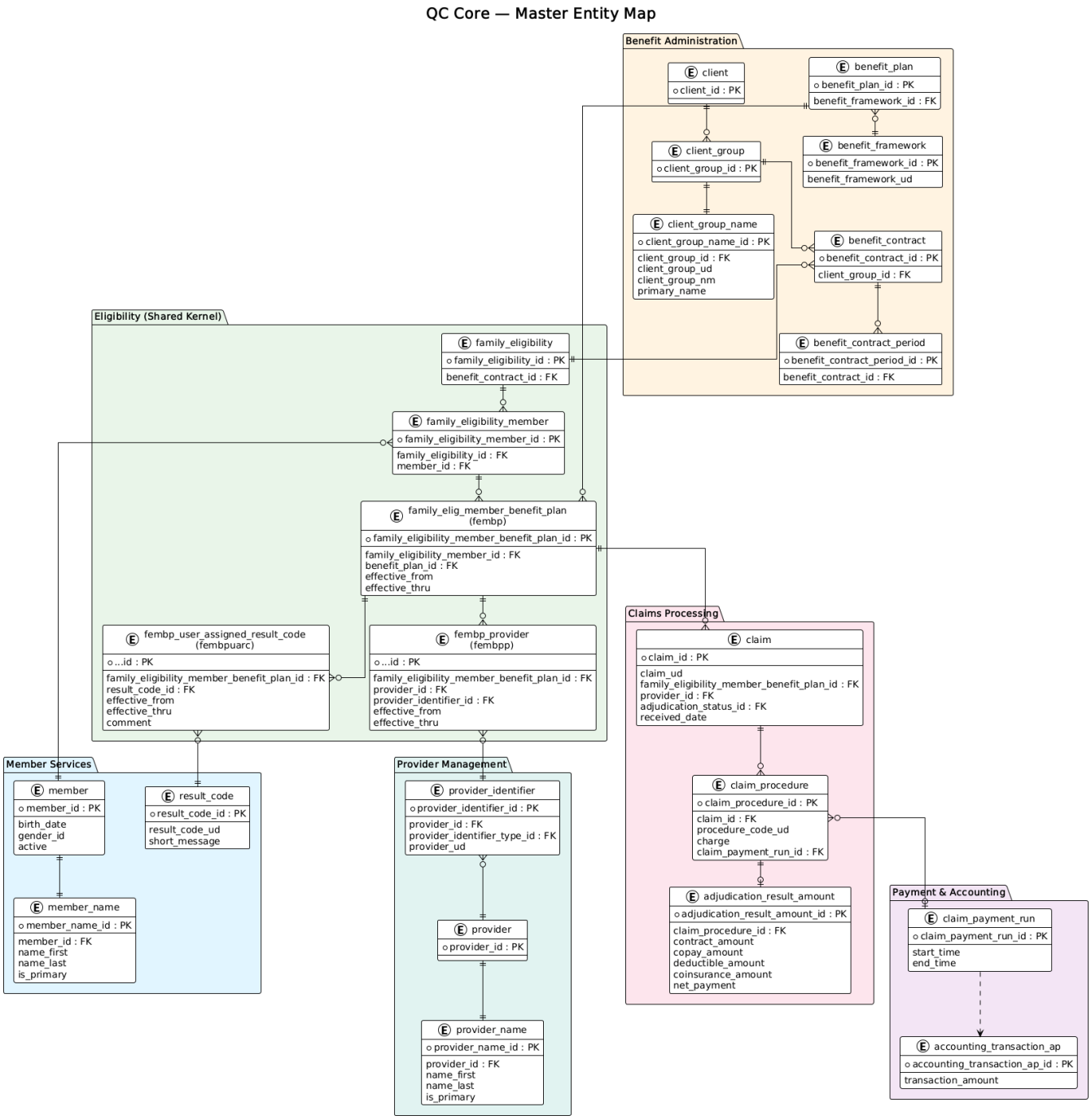

Figure 2: Master Entity Relationship Diagram

Key insight: The `family_eligibility_member_benefit_plan` table (abbreviated `fembp` everywhere) is the **nexus of the entire system**. It answers "which member has which plan" and is the FK target for claims, provider assignments, result codes, and referrals.

Act 1: Enrollment — "I Got My Insurance Card"

The Real-World Story

Maria Garcia-TRAIN starts a new job at Acme Manufacturing. During orientation, HR tells her the company offers health insurance through Verda Health Plan. Maria fills out enrollment forms, choosing the "HMO Gold" plan. She picks coverage for herself, her husband Carlos, and their daughter Sofia. Two weeks later, she receives insurance ID cards in the mail.

> **Try it now:** Every entity below exists in your local qc_core. Search for `Garcia-TRAIN` in the member module, or run: `SELECT * FROM member_name WHERE name_last = 'Garcia-TRAIN'`

What Just Happened in qc_core

Every entity in Maria's story maps to a specific table — and to a real row you can query:

Real World	qc_core Table	Seeded Value
Verda Health Plan (the TPA)	`client` / `client_name`	client_ud = `TRAIN_VERDA`
Acme Manufacturing (employer)	`client_group` / `client_group_name`	client_group_ud = `TRAIN_ACME`
The insurance contract	`benefit_contract`	benefit_contract_ud = `TRAIN_ACME_2026`
Contract period (Jan-Dec 2026)	`benefit_contract_period`	effective_from = 2026-01-01
"HMO Gold" plan	`benefit_plan` / `benefit_framework`	benefit_framework_ud = `TRAIN_HMO_GOLD`
The Garcia household	`family_eligibility`	benefit_contract_id -> TRAIN_ACME_2026
Maria (subscriber)	`family_eligibility_member`	relationship_to_insured = Self
Carlos (spouse)	`family_eligibility_member`	relationship_to_insured = Spouse
Sofia (child)	`family_eligibility_member`	relationship_to_insured = Child
Maria enrolled in HMO Gold	`family_eligibility_member_benefit_plan`	effective_from = 2026-01-01, effective_thru = NULL
Maria herself	`member` / `member_name`	name_first = `Maria`, name_last = `Garcia-TRAIN`

ER Diagram: The Enrollment Chain

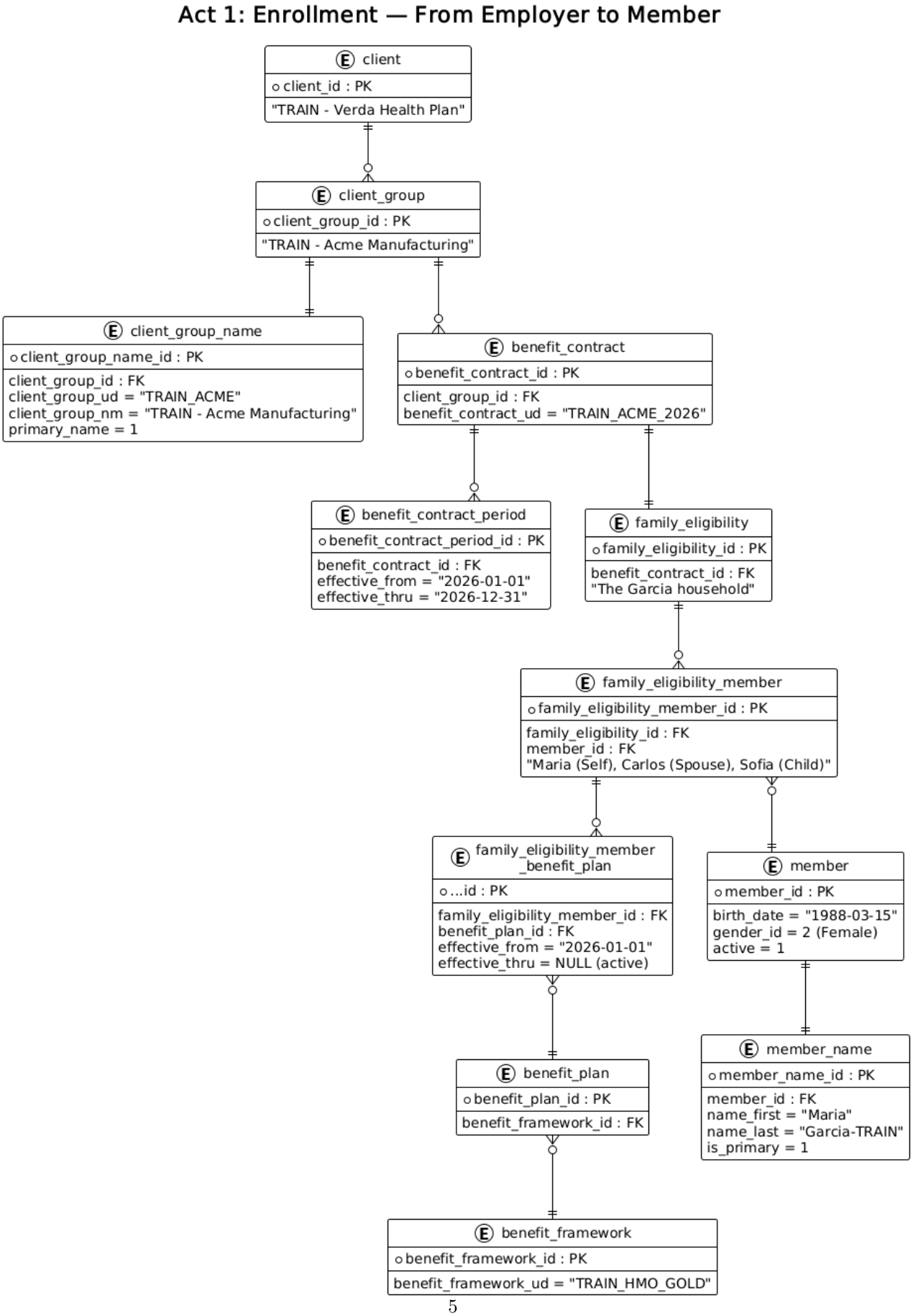

Figure 3: Enrollment Chain: From Employer to Member

Plexis Benefits Chain (from training deck)

This is the official Plexis ER diagram showing the benefits hierarchy from Client down to Treatment Types:

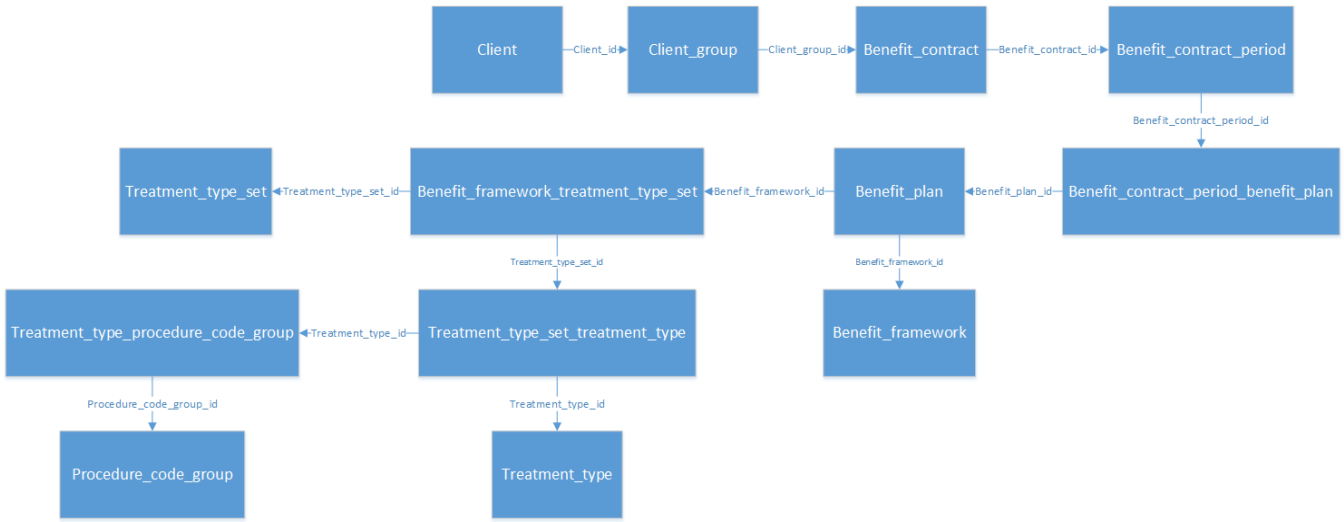

Figure 4: Plexis Benefits Chain ER

Reading It Top-Down

1. **client** is the health plan organization (`TRAIN - Verda Health Plan`)
2. **client_group** is the employer (`TRAIN - Acme Manufacturing`) — one client can have many groups
3. **benefit_contract** is the legal agreement (`TRAIN_ACME_2026`)
4. **benefit_contract_period** is the annual coverage window (2026-01-01 to 2026-12-31)
5. **family_eligibility** is the Garcia household — linked to the contract
6. **family_eligibility_member** links each person (Maria=Self, Carlos=Spouse, Sofia=Child) to that household
7. **family_eligibility_member_benefit_plan** is the golden record: Maria is enrolled in TRAIN_HMO_GOLD, effective 2026-01-01, no end date (active)

Try It Yourself — SQL Walkthrough

Run this against your local qc_core. You should see Maria, Carlos, and Sofia:

```sql
-- Walk the enrollment chain for the Garcia family
SELECT
    m.member_id,
    mn.name_first + ' ' + mn.name_last AS member_name,
    ri.relationship_to_insured_ud       AS relationship,
    fe.family_eligibility_id            AS family_id,
    bc.benefit_contract_ud              AS contract,
    cgn.client_group_nm                 AS employer_group,
    bf.benefit_framework_ud             AS plan_type,
    fembp.effective_from                AS coverage_start,
    fembp.effective_thru                AS coverage_end
FROM member m
JOIN member_name mn ON m.member_id = mn.member_id AND mn.is_primary = 1
JOIN family_eligibility_member fem ON m.member_id = fem.member_id
JOIN relationship_to_insured ri ON fem.relationship_to_insured_id = ri.relationship_to_insured_id
JOIN family_eligibility fe ON fem.family_eligibility_id = fe.family_eligibility_id
JOIN family_eligibility_member_benefit_plan fembp
    ON fem.family_eligibility_member_id = fembp.family_eligibility_member_id
JOIN benefit_plan bp ON fembp.benefit_plan_id = bp.benefit_plan_id
JOIN benefit_framework bf ON bp.benefit_framework_id = bf.benefit_framework_id
```

```
JOIN benefit_contract bc ON fe.benefit_contract_id = bc.benefit_contract_id
JOIN client_group cg ON bc.client_group_id = cg.client_group_id
JOIN client_group_name cgn ON cg.client_group_id = cgn.client_group_id
    AND cgn.primary_name = 1
WHERE mn.name_last = 'Garcia-TRAIN'
ORDER BY mn.name_first;
```

Expected results: Maria (Self) and Carlos (Spouse) each have a TRAIN_HMO_GOLD enrollment. Sofia is in the family but enrolled as a dependent through Maria's plan.

The QC Member Screen

This is what the QC Member Edit screen looks like. Notice the tabs along the top: General, Eligibility, Medical Conditions, Linked Members, Addresses, Contacts, Communications, and User Assigned Result Codes. Each tab maps to a different set of tables.

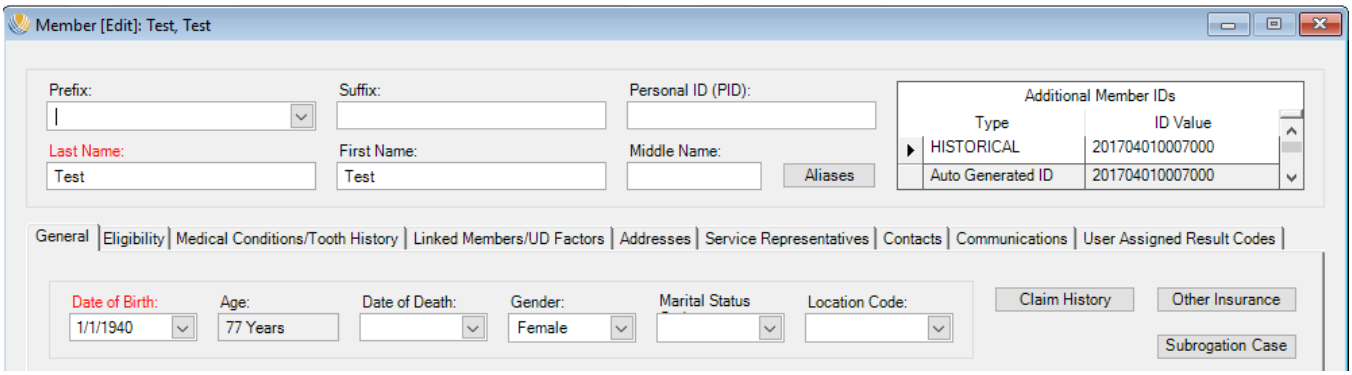

Figure 5: QC Member Edit Screen

Plexis Member ER (from training deck)

The official Plexis diagram showing the member entity and its child tables:

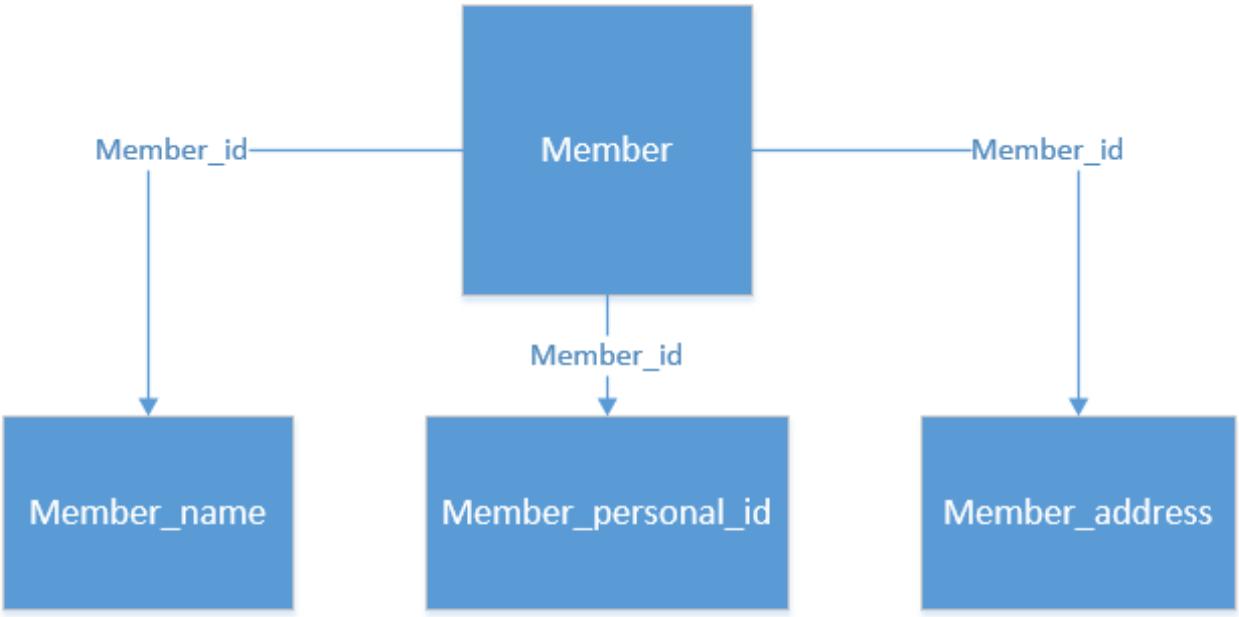

Figure 6: Plexis Member ER

Plexis Eligibility Chain (from training deck)

The official Plexis diagram showing the eligibility join chain – the most important relationship in qc_core:

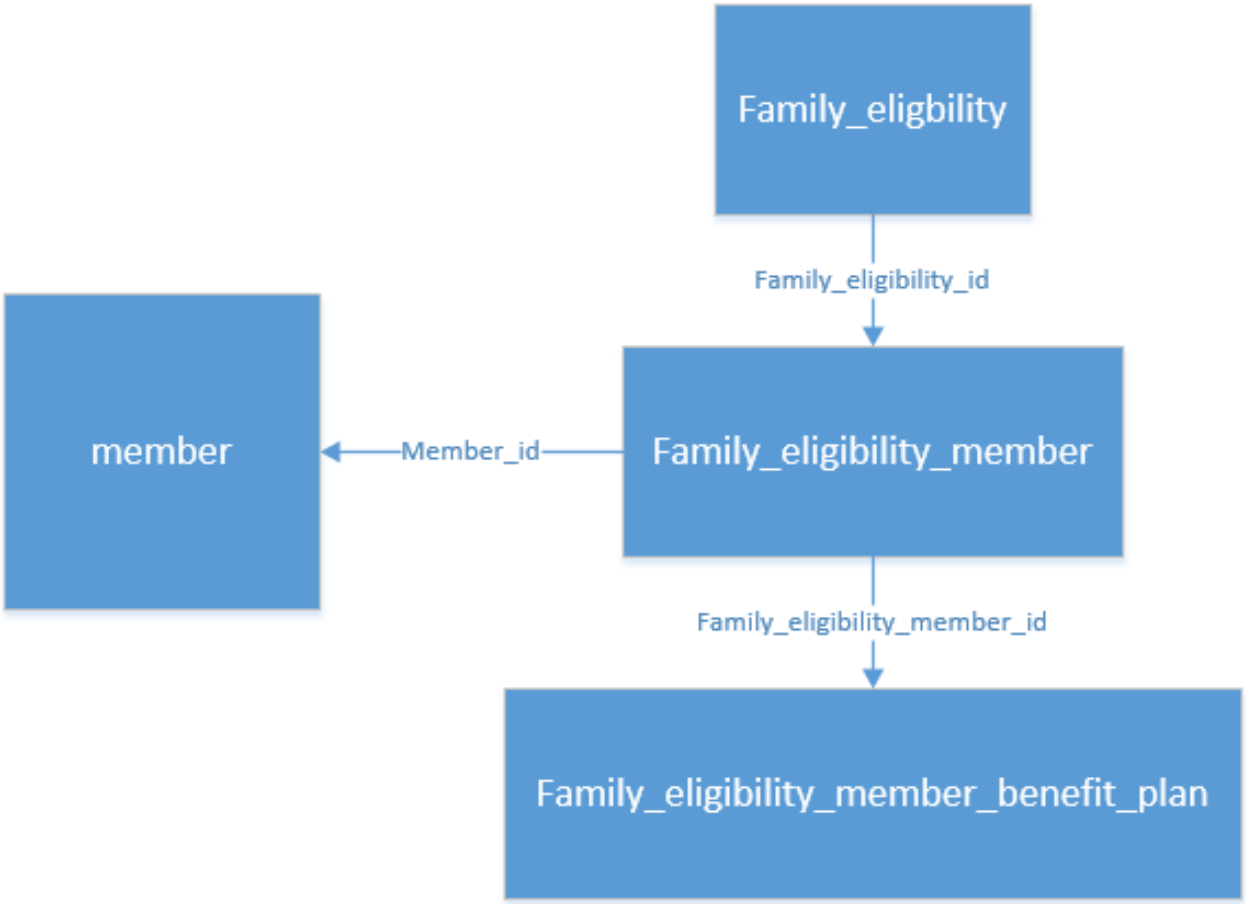

Figure 7: Plexis Eligibility Chain ER

Benefits to Eligibility Link (from training deck)

This shows the two distinct paths linking benefits to eligibilities. The left side is the enrollment chain, the right side is the contract-period chain. They meet at `family_eligibility_member_benefit_plan`:

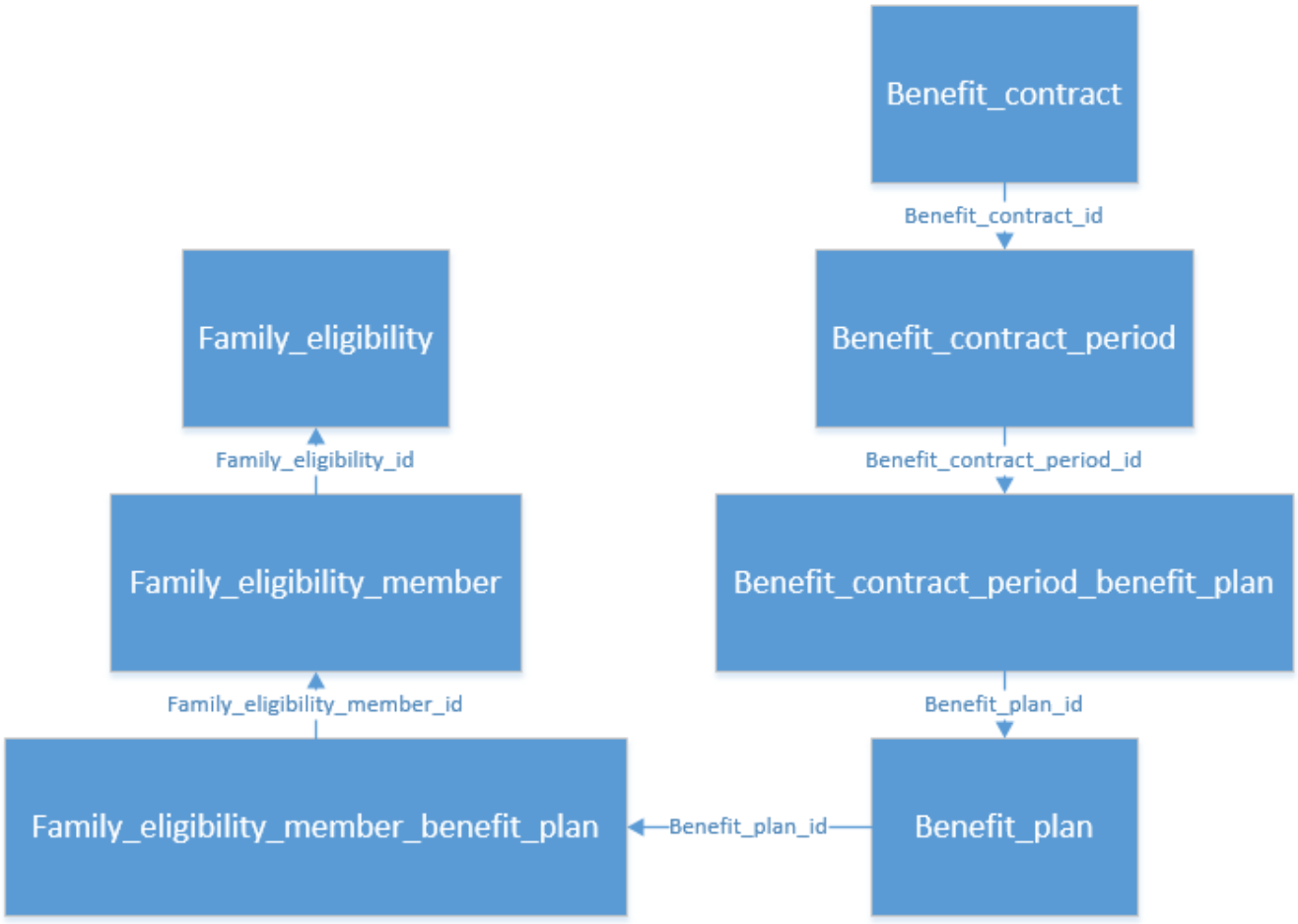

Figure 8: Plexis Benefits to Eligibility ER

Act 2: Choosing a Doctor — "Who's My PCP?"

The Real-World Story

Maria's HMO Gold plan requires her to choose a Primary Care Provider (PCP). She browses the provider directory and picks Dr. Carlos Rodriguez-TRAIN, who belongs to the Vicare IPA (Independent Practice Association). The IPA is a network of doctors who have contracted with the health plan.

Dr. Rodriguez's provider ID is `VIC40001` — the "VIC" prefix identifies his IPA network.

> **Try it now:** `SELECT * FROM provider_name WHERE name_last = 'Rodriguez-TRAIN'`

What Just Happened in qc_core

Real World	qc_core Table	Seeded Value
Dr. Rodriguez	`provider` / `provider_name`	name_last = `Rodriguez-TRAIN`
His NPI identifier	`provider_identifier`	provider_ud = `VIC40001`, type = NPI
Maria's PCP assignment	`family_eligibility_member_benefit_plan.pcp_provider`	effective_from = 2026-01-15, effective_thru = NULL
Vicare IPA network	Derived from `LEFT(provider_ud, 3) = 'VIC'`	IPA identified by prefix

ER Diagram: Provider & PCP Assignment

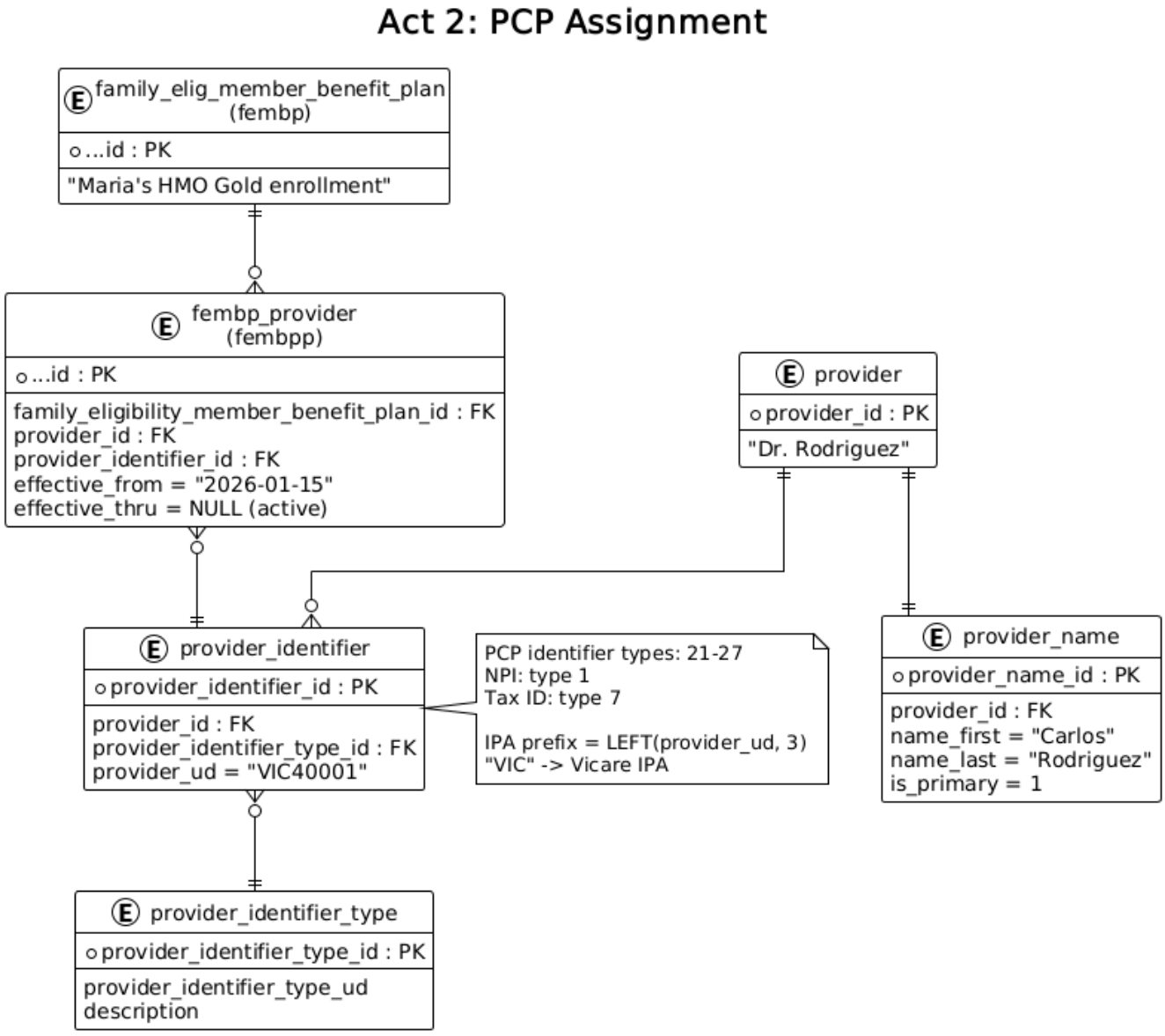

Figure 9: Provider and PCP Assignment

The IPA Prefix System

In managed care (HMO plans), providers are organized into IPAs. In qc_core, the IPA isn't a separate table — it's encoded in the first 3 characters of the `provider_ud` field on `provider_identifier`:

Prefix	IPA Name	QCAP Result Code
VIC	Vicare	QCAPVICARE
GEN	Genesis Medical Group	QCAPGMG
HOU	Houston Physicians IPA	QCAPHPIPA
TPH	TX Physicians Health Net	QCAPPNS
INT	INet IPA	QCAPINET
OCI	OneCare IPA	QCAPOCIPA
CMR	CareMore IPA	QCAPCM

Prefix	IPA Name	QCAP Result Code
PHP	Phoenix Physicians IPA	QCAPPPIPA

Why this matters: When Maria picks Dr. Rodriguez (prefix VIC), the system needs to tag her benefit plan with the result code `QCAPVICARE`. This is what the weekly QCAP sync does.

Try It Yourself — SQL Walkthrough

```sql
-- Find Maria's PCP and their IPA
SELECT
    m.member_id,
    mn.name_first + ' ' + mn.name_last AS member_name,
    pn.name_first + ' ' + pn.name_last AS pcp_name,
    pid.provider_ud AS pcp_identifier,
    LEFT(pid.provider_ud, 3) AS ipa_prefix,
    fembpp.effective_from AS pcp_assigned_date,
    fembpp.effective_thru AS pcp_termed_date
FROM member m
JOIN member_name mn ON m.member_id = mn.member_id AND mn.is_primary = 1
JOIN family_eligibility_member fem ON m.member_id = fem.member_id
JOIN family_eligibility_member_benefit_plan fembp
    ON fem.family_eligibility_member_id = fembp.family_eligibility_member_id
JOIN family_eligibility_member_benefit_plan_provider fembpp
    ON fembp.family_eligibility_member_benefit_plan_id
       = fembpp.family_eligibility_member_benefit_plan_id
JOIN provider_identifier pid
    ON pid.provider_identifier_id = fembpp.provider_identifier_id
LEFT JOIN provider_name pn
    ON fembpp.provider_id = pn.provider_id AND pn.is_primary = 1
WHERE mn.name_last = 'Garcia-TRAIN';
```

Expected result: Maria Garcia-TRAIN -> Dr. Carlos Rodriguez-TRAIN, PCP identifier VIC40001, IPA prefix VIC, assigned 2026-01-15, active (termed = NULL).

Act 3: The QCAP Sync — "Tagging Members to Their IPA"

The Real-World Story

Every week, Verda's system needs to answer: "Does every member's benefit plan have the correct QCAP result code for their current PCP's IPA?" This matters because downstream processes (claims adjudication, capitation payments, reporting) rely on these codes to route work correctly.

Three things can happen:

1. **NEW member/PCP** — Maria just enrolled and picked Dr. Rodriguez (VIC prefix). She needs the `QCAPVICARE` code assigned.
2. **CHANGED PCP** — Maria's old doctor Dr. Chen (GEN prefix, `QCAPGMG`) left the network. She switched to Dr. Rodriguez (VIC prefix). The old `QCAPGMG` code needs to be terminated and `QCAPVICARE` inserted.
3. **UNCHANGED** — Maria still has Dr. Rodriguez, and her `QCAPVICARE` code is current. Do nothing.

This is exactly what `weekly-qcap-sync.sql` (RT-27730 v4) automates.

Use Case Diagram

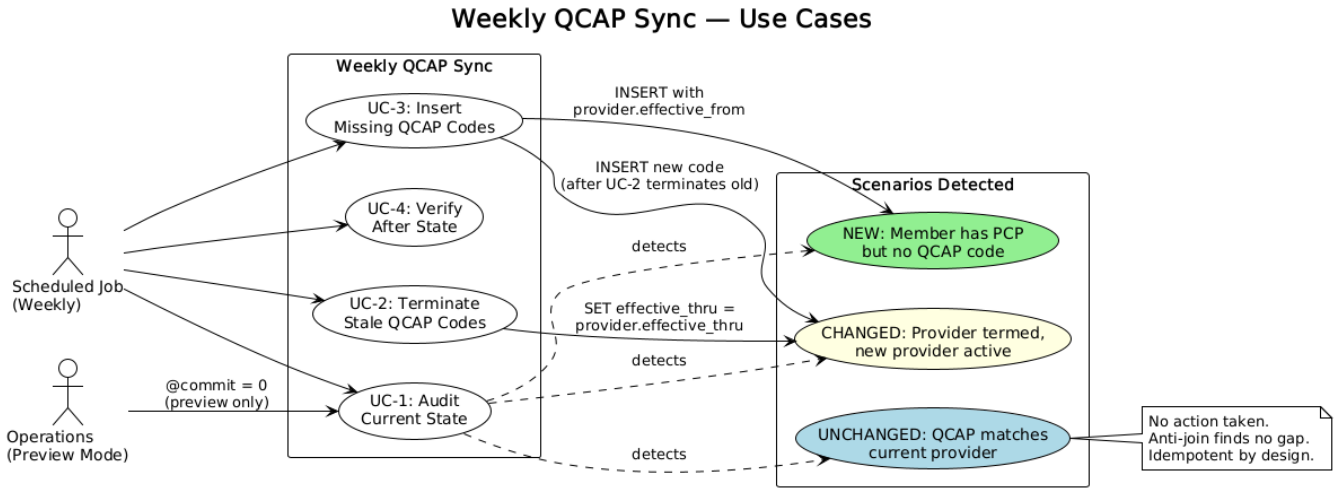

Figure 10: Weekly QCAP Sync Use Cases

Activity Diagram: The Weekly Sync Flow

This maps directly to the four PARTs of `weekly-qcap-sync.sql`:

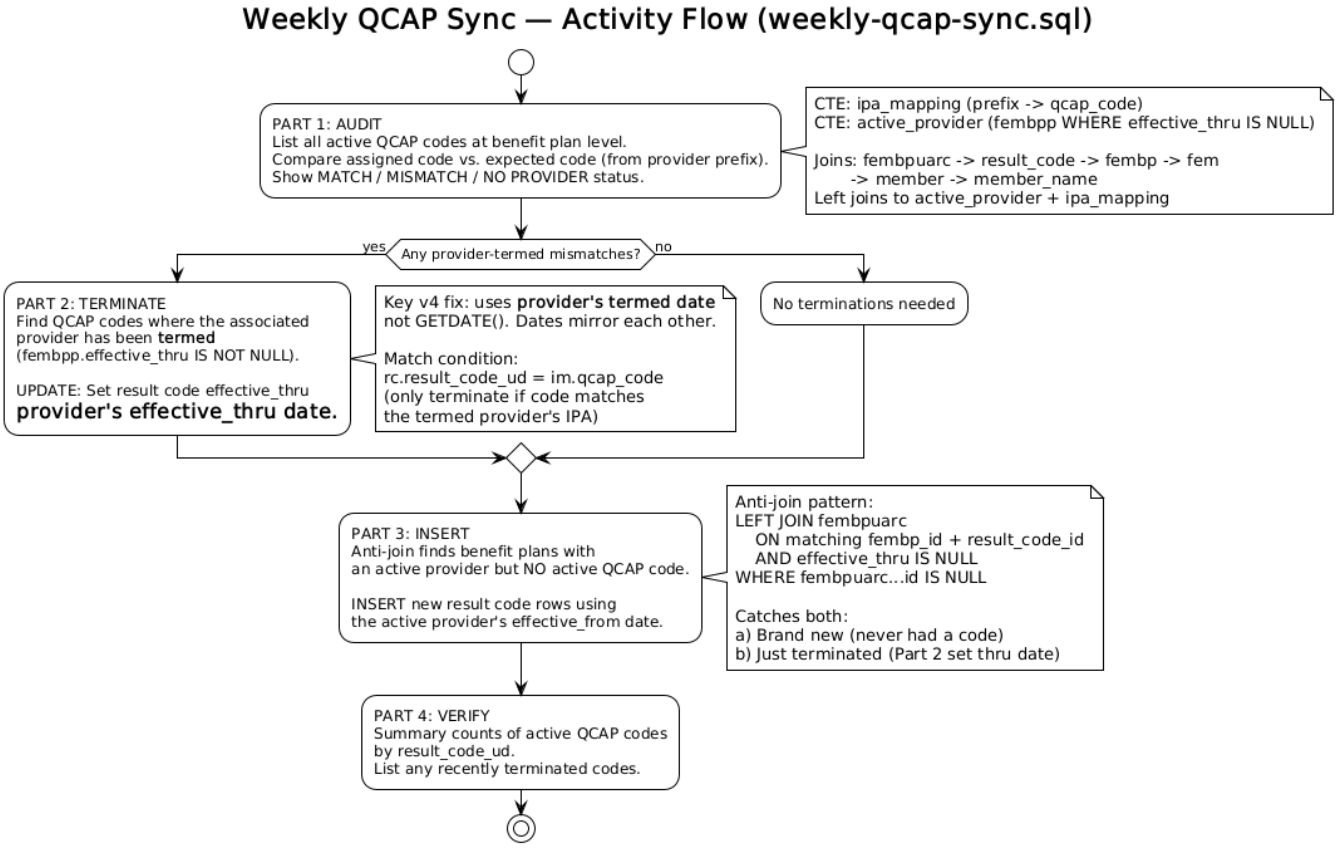

Figure 11: Weekly QCAP Sync Activity Flow

Sequence Diagram: The Anti-Join Pattern (Gap Detection)

The anti-join is the core technique. Here's how it finds members who need a QCAP code:

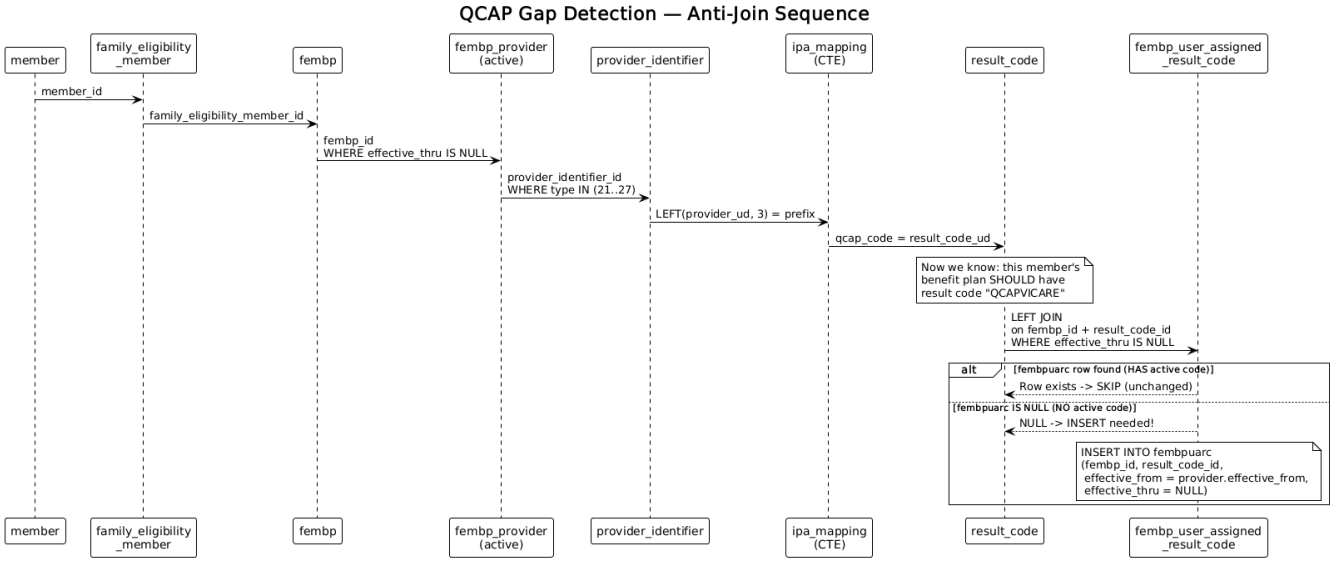

Figure 12: QCAP Gap Detection: Anti-Join Sequence

ER Diagram: The QCAP Sync Tables

These are the specific tables the sync script touches:

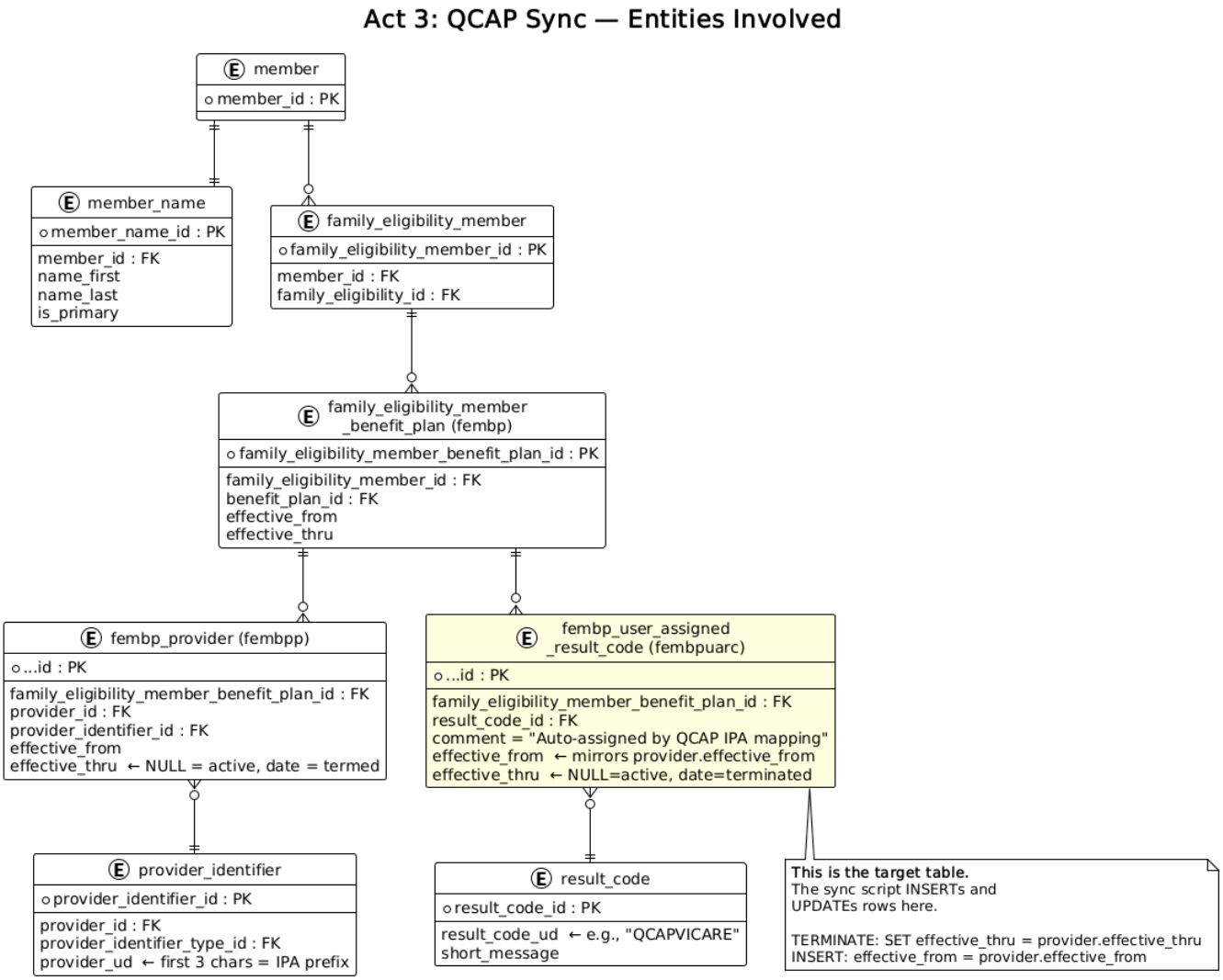

Figure 13: QCAP Sync Entities

The Three Scenarios in Detail

Scenario 1: NEW — First-Time Assignment

```
BEFORE:  fembp has provider (VIC prefix) but NO row in fembpuarc for QCAPVICARE
AFTER:   INSERT into fembpuarc with effective_from = provider.effective_from
```

Maria just enrolled and got assigned to Dr. Rodriguez (VIC). The anti-join in Part 3 detects that her benefit plan has no active `QCAPVICARE` code. It inserts one, using Dr. Rodriguez's assignment date as the effective_from.

Scenario 2: CHANGED — Provider Switch

```
BEFORE:  fembpuarc has active QCAPGMG (from old Dr. Chen, GEN prefix)
         fembpp has old provider with effective_thru = 2026-03-15
         fembpp has new provider (VIC prefix) with effective_thru = NULL

STEP 1 (Part 2 TERMINATE):
         UPDATE fembpuarc SET effective_thru = 2026-03-15
         (the old provider's termed date - dates mirror)

STEP 2 (Part 3 INSERT):
```

```
        Anti-join now finds NO active QCAPVICARE for this fembp
        INSERT new QCAPVICARE with effective_from = new provider.effective_from
```

Maria switched doctors. The sync first terminates the old code using the old provider's termed date, then the anti-join picks up the gap and inserts the new code.

Scenario 3: UNCHANGED — No Action

```
BEFORE:  fembpuarc has active QCAPVICARE
         fembpp has active VIC-prefix provider

RESULT:  Part 2 skips (no termed provider matching this code)
         Part 3 skips (anti-join finds existing active code)
```

Everything is in sync. The script is idempotent — running it again changes nothing.

Why This Matters in the Real World

QCAP result codes drive critical downstream processes:

1. **Capitation payments** — The health plan pays IPAs a monthly per-member fee. The QCAP code determines which IPA gets paid for which member.
2. **Claims routing** — When a claim comes in, the QCAP code helps determine which provider network should handle adjudication.
3. **Reporting** — Member counts by IPA, provider utilization reports, and financial reconciliation all depend on accurate QCAP assignment.

If a member's QCAP code is wrong or missing, the wrong IPA gets paid, claims may adjudicate incorrectly, and reports show inaccurate numbers.

Act 4: Getting a Referral — "I Need to See a Specialist"

The Real-World Story

Maria has been having knee pain. Dr. Rodriguez-TRAIN examines her and decides she needs to see an orthopedic specialist. Because Maria's HMO Gold plan requires referrals, Dr. Rodriguez submits a referral authorization. The health plan approves it for 3 visits to Dr. Jennifer Park-TRAIN (the orthopedist) over 90 days (2026-04-10 to 2026-07-10).

> **Try it now:** `SELECT * FROM referral WHERE referral_ud = 'TRAIN-REF-001'`

What Just Happened in qc_core

Real World	qc_core Table	What It Stores
The referral itself	`referral`	Authorization record with dates, limits
Dr. Rodriguez (referring)	`referral.referring_provider_id`	FK to provider
Dr. Park (specialist)	`referral_provider`	Authorized provider(s)
"3 visits, 90 days"	`referral.maximum_visits`, `effective_thru`	Usage limits
Approved knee procedures	`referral_authorized_procedure`	What services are approved
Referral status	`referral_status`	Pending, Approved, Denied, etc.
Link to Maria's plan	`referral.family_eligibility_member_benefit_plan_id`	FK to benefit plan

ER Diagram: Referral Structure

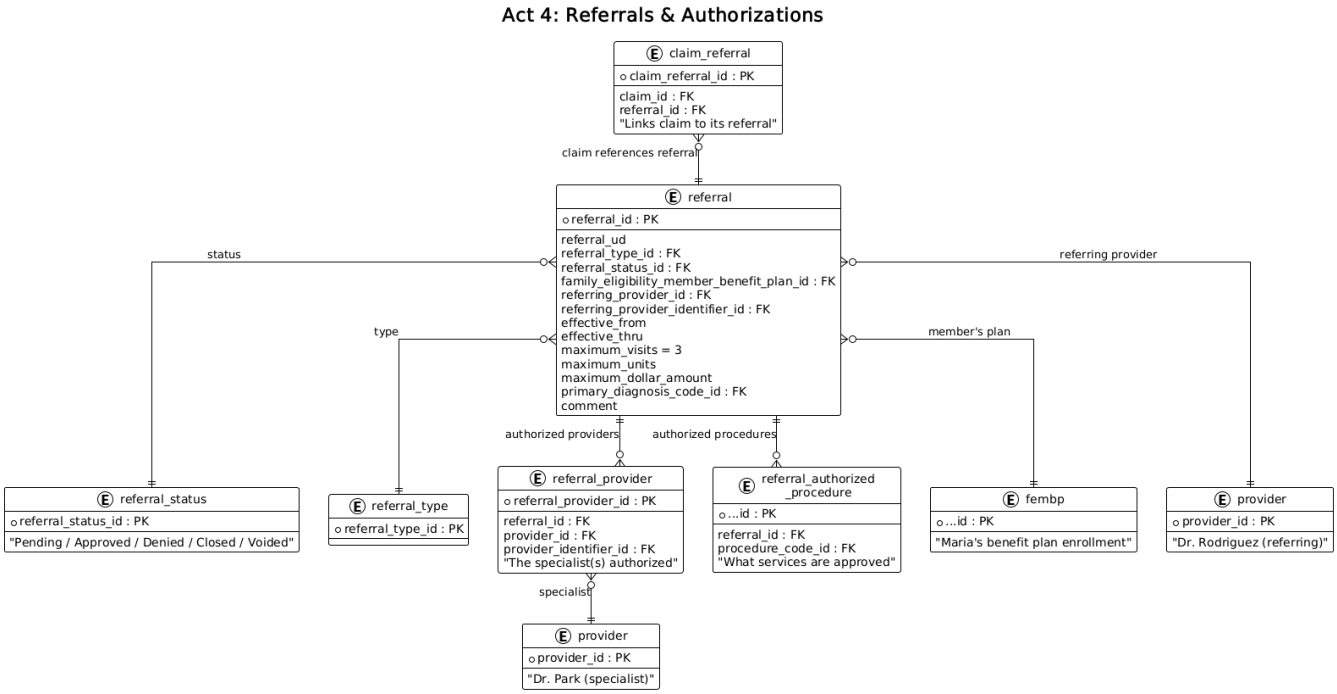

Figure 14: Referrals and Authorizations

The Referral-to-Claim Link

When Maria later visits Dr. Park and a claim is submitted, the claim is linked back to the referral through `claim_referral`. During adjudication, the system checks: was this visit authorized? Is it within the visit limit? Is it before the expiration date?

Act 5: The Visit & Claim — "The Doctor Bills Insurance"

The Real-World Story

Maria visits Dr. Park-TRAIN for her knee on 2026-04-15. At check-in, she shows her insurance card. Dr. Park examines her (CPT code 99201 – office visit, new patient) and takes X-rays (CPT code 73560 – knee X-ray). The office submits an electronic claim (`TRAIN-CLM-001`) to Verda Health Plan.

> **Try it now:** `SELECT * FROM claim WHERE claim_ud = 'TRAIN-CLM-001'`

The claim has a **header** (who, when, where) and **line items** (what services, how much). The health plan's adjudication engine processes it automatically: checks eligibility, applies benefit rules, calculates what each party owes.

What Just Happened in qc_core

Claim header:

Real World	qc_core Column	Seeded Value
Claim tracking number	`claim.claim_ud`	`TRAIN-CLM-001`
Maria's enrollment	`claim.family_eligibility_member_benefit_plan_id`	FK to Maria's fembp
Dr. Park (rendering)	`claim.provider_id`	FK to Park-TRAIN
Billing entity	`claim.billing_provider_id`	FK to Park-TRAIN
Date received	`claim.received_date`	2026-04-15
Clean claim date	`claim.clean_date`	2026-04-15

Real World	qc_core Column	Seeded Value
Status	`claim.adjudication_status_id`	1 = Approved
Form type	`claim.claim_form_type_id`	3 = Professional (CMS-1500)

Claim line items:

Real World	qc_core Column	Line 1	Line 2
Procedure code	`claim_procedure.procedure_code_ud`	99201 (Office Visit)	73560 (Knee X-Ray)
Charge	`claim_procedure.charge`	$300.00	$150.00
Service date	`claim_procedure.service_from_date`	2026-04-15	2026-04-15
System generated?	`claim_procedure.is_system_generated`	0 (real)	0 (real)

ER Diagram: Claim & Adjudication

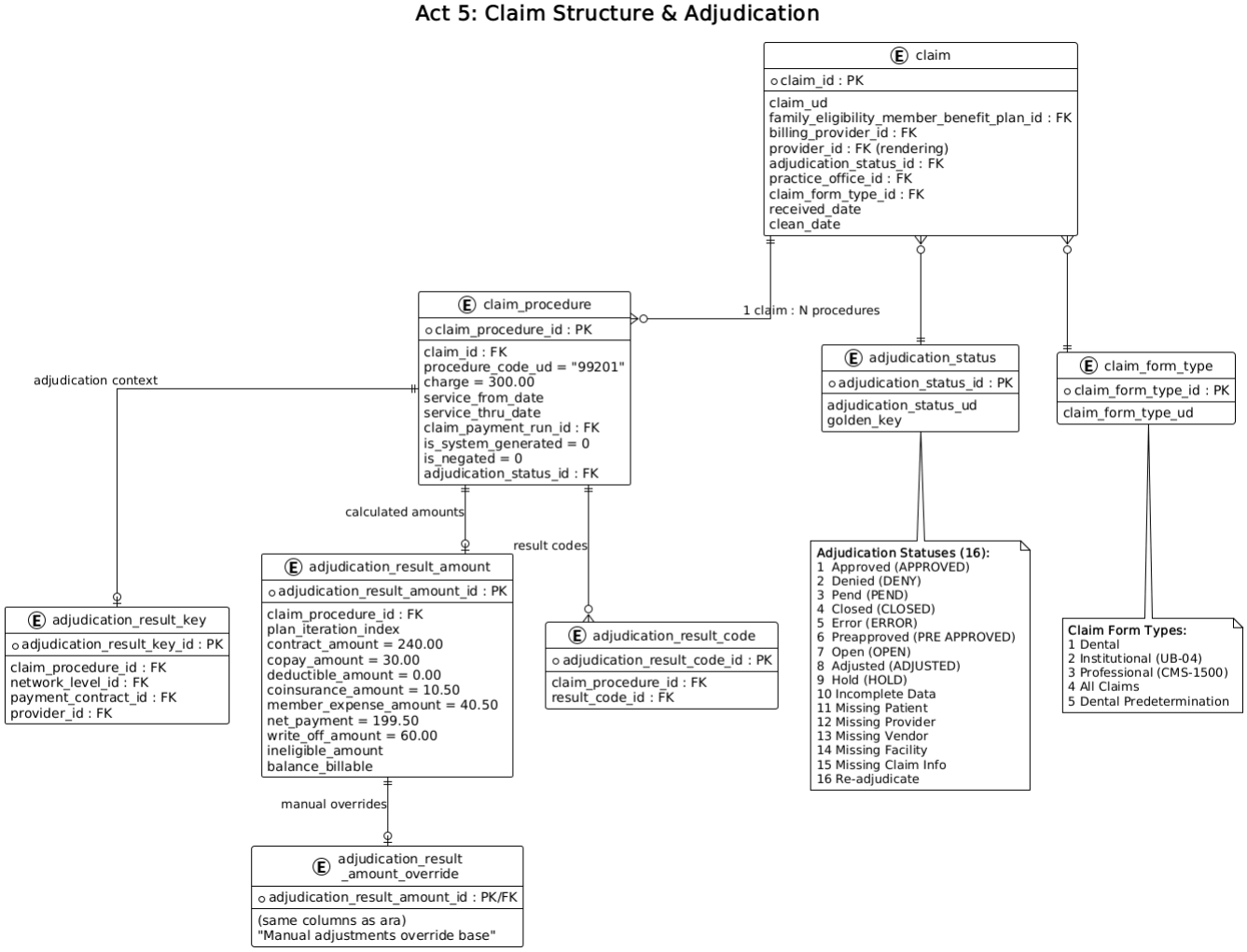

Figure 15: Claim Structure and Adjudication

The Adjudication Math

Here's what the adjudication engine calculated for Maria's two claim lines. You can verify every number against the seeded data.

Line 1: Office Visit (99201, $300 charge):

```
  Billed charge:                    $300.00
- Write-off (contract discount):    - $60.00   (provider agreed to accept less)
= Contract amount:                  $240.00   (what the plan considers)

  Contract amount:                  $240.00
- Copay (Maria's flat fee):         - $30.00
- Deductible (already met):         - $0.00
- Coinsurance (Maria's 5%):         - $10.50
= Net payment (plan pays provider): $199.50

  Maria's responsibility:           $40.50    (copay + coinsurance)
```

Line 2: Knee X-Ray (73560, $150 charge):

```
  Billed charge:                    $150.00
- Write-off:                        - $25.00
= Contract amount:                  $125.00

  Contract amount:                  $125.00
- Copay:                            - $0.00    (copay only on office visit)
- Deductible:                       - $0.00
- Coinsurance (5%):                 - $6.25
= Net payment:                      $118.75

  Maria's responsibility:           $6.25
```

Totals: Plan pays Dr. Park $318.25. Maria owes $46.75.

> **Try it now:** Run the SQL walkthrough below and verify these exact numbers.

In the database for Line 1: - `adjudication_result_amount.contract_amount` = 240.00 - `adjudication_result_amount.copay_` = 30.00 - `adjudication_result_amount.coinsurance_amount` = 10.50 - `adjudication_result_amount.net_payment` = 199.50 - `adjudication_result_amount.write_off_amount` = 60.00 - `adjudication_result_amount.member_expense_amoun` = 40.50

Override pattern: If someone manually adjusts amounts, the override table takes precedence:

```
COALESCE(override.net_payment, base.net_payment, 0)
```

Try It Yourself — SQL Walkthrough

```
-- See Maria's claim with its adjudication amounts
SELECT
    c.claim_ud,
    adj_s.adjudication_status_ud AS claim_status,
    cp.procedure_code_ud         AS proc_code,
    cp.charge,
    ara.contract_amount,
    ara.copay_amount,
    ara.deductible_amount,
    ara.coinsurance_amount,
    ara.net_payment,
    ara.write_off_amount,
    ara.member_expense_amount    AS maria_owes
FROM claim c
JOIN adjudication_status adj_s
    ON c.adjudication_status_id = adj_s.adjudication_status_id
JOIN claim_procedure cp
    ON c.claim_id = cp.claim_id
```

```
    AND cp.is_system_generated = 0
    AND cp.charge > 0
LEFT JOIN adjudication_result_amount ara
    ON cp.claim_procedure_id = ara.claim_procedure_id
WHERE c.claim_ud = 'TRAIN-CLM-001'
ORDER BY cp.procedure_code_ud;
```

Expected results:

claim_ud	status	proc_code	charge	contract	copay	net_payment	maria_owes
TRAIN-CLM-001	Approved	73560	150.00	125.00	0.00	118.75	6.25
TRAIN-CLM-001	Approved	99201	300.00	240.00	30.00	199.50	40.50

Plexis Claims ER (from training deck)

The official Plexis diagram showing how claim types, procedures, and adjudication tables relate:

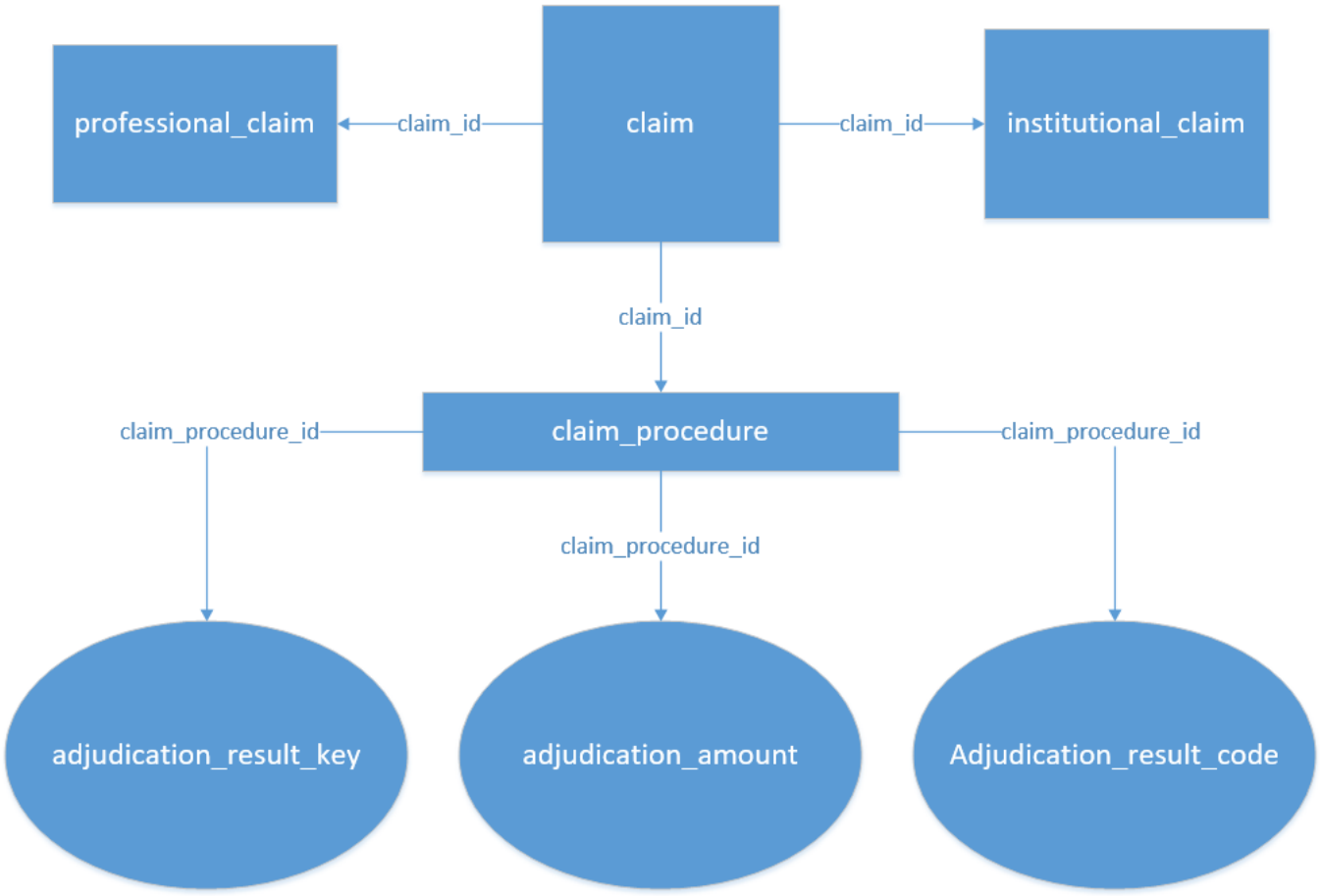

Figure 16: Plexis Claims ER

Key Business Rules

1. **Filter real procedures:** `is_system_generated = 0 AND charge > 0` (optionally `is_negated = 0`)
2. **Override precedence:** `COALESCE(override_value, base_value, 0)` for most amounts

3. **Multi-plan:** `plan_iteration_index` tracks base vs. supplemental plan processing
4. **Count claims, not lines:** `COUNT(DISTINCT claim_id)` prevents over-counting from the 1:N claim-to-procedure relationship

---

Act 6: Payment — "The Provider Gets Paid"

The Real-World Story

After adjudication, the health plan batches approved claims into payment runs. Dr. Park-TRAIN's payment ($318.25 total across both lines) is included in payment run `TRAIN-PAYRUN-001` on 2026-04-22. The plan sends Dr. Park an electronic payment and sends Maria an Explanation of Benefits (EOB) showing what was paid and what she owes ($46.75).

> **Try it now:** `SELECT * FROM claim_payment_run WHERE claim_payment_run_ud = 'TRAIN-PAYRUN-001'`

What Just Happened in qc_core

Real World	qc_core Table	What It Stores
Payment batch	`claim_payment_run`	Batch with start/end timestamps
"Date paid"	`claim_payment_run.start_time`	**Authoritative** paid date
Link to procedure	`claim_procedure.claim_payment_run_id`	Which run paid this line
A/P transaction	`accounting_transaction_ap`	The actual payment record
A/P claim detail	`accounting_transaction_ap_claim_detail`	Links A/P to claim procedure
Payment method	`accounting_transaction_ap_payment_detail`	Check date, EFT info
A/P apply-to chain	`accounting_transaction_ap_apply_to`	Payment matching

ER Diagram: Payment Flow

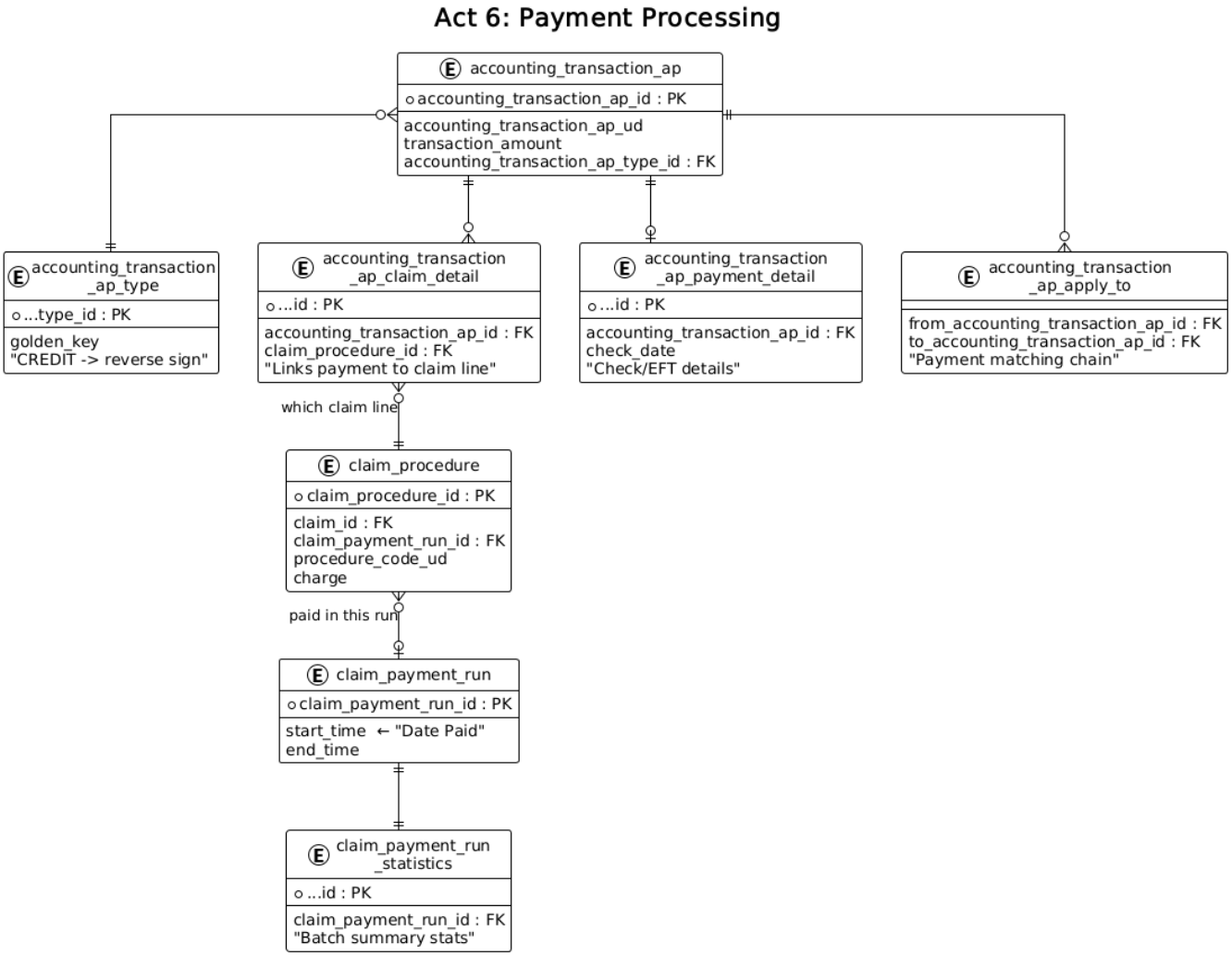

Figure 17: Payment Processing Flow

Plexis A/P Accounting ER (from training deck)

The official Plexis diagram showing the payment flow from claim_payment_run through to AP transactions:

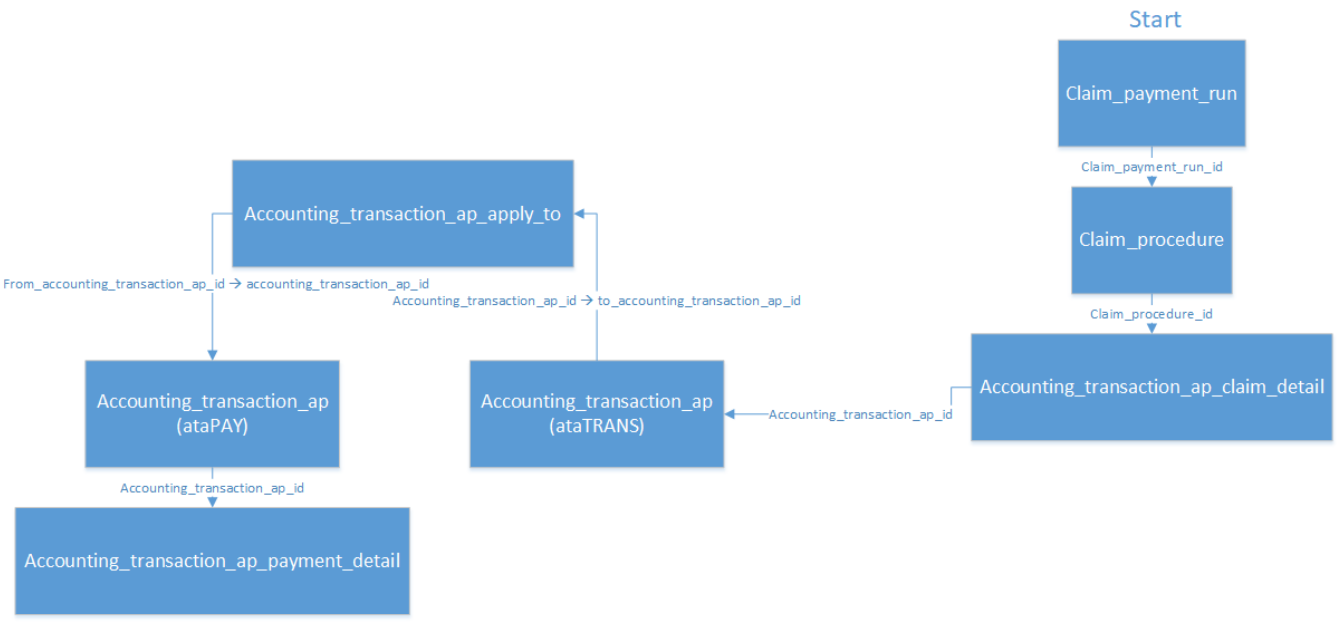

Figure 18: Plexis AP Accounting ER

The A/R Side (Premium Billing)

On the other side of the ledger, the health plan bills employers for premiums:

Real World	qc_core Table
Premium invoice	`accounting_transaction_ar`
Premium detail	`accounting_transaction_ar_premium_detail`
Invoice detail	`accounting_transaction_ar_invoice_detail`
Payment received	`accounting_transaction_ar_payment_detail`

The A/R side tracks money coming IN (premiums from employers), while A/P tracks money going OUT (payments to providers).

Business Rules

1. **Date paid** = `claim_payment_run.start_time` (not end_time, not check_date)
2. **Date filtering:** `start_time >= @from AND start_time < DATEADD(DAY, 1, @thru)` (open-ended)
3. **Payment is line-level:** `claim_payment_run` links to `claim_procedure`, not `claim`
4. **Credit handling:** When `accounting_transaction_ap_type.golden_key = 'CREDIT'`, reverse the sign: `-1 * ABS(amount)`
5. **EOB print date fallback:** `COALESCE(check_date, last_printed_date, NULL)`

End-to-End Use Case Diagram

This shows the complete lifecycle from the patient's perspective:

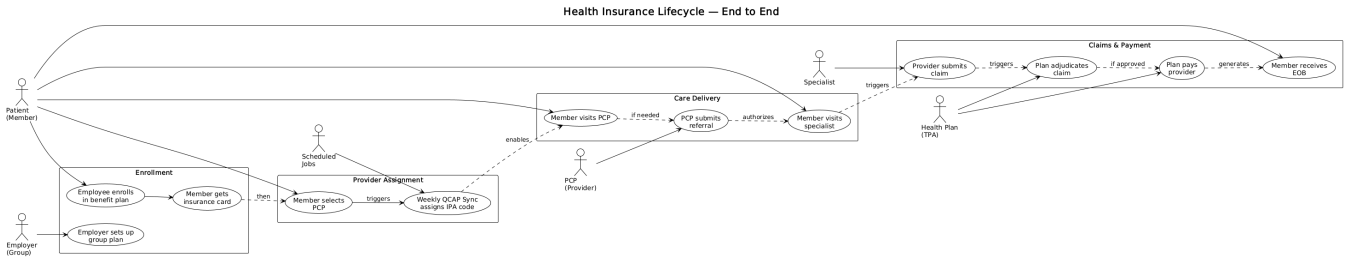

Figure 19: End-to-End Insurance Lifecycle

Quick Reference: The Two Critical Join Chains

Almost every report and stored procedure in qc_core uses one of these two join chains.

The Eligibility Chain (Member to Provider)

"Given a member, find their PCP."

```
member
  -> family_eligibility_member            (ON member_id)
  -> family_eligibility_member_benefit_plan  (ON family_eligibility_member_id)
  -> family_eligibility_member_benefit_plan_provider  (ON fembp_id)
  -> provider_identifier                  (ON provider_identifier_id)
  -> provider                             (ON provider_id)
```

```sql
-- Eligibility chain: Member to PCP (try with Garcia-TRAIN)
SELECT m.member_id, mn.name_first, mn.name_last,
       pn.name_last AS pcp_name, pid.provider_ud
FROM member m
JOIN member_name mn
    ON m.member_id = mn.member_id AND mn.is_primary = 1
JOIN family_eligibility_member fem
    ON m.member_id = fem.member_id
JOIN family_eligibility_member_benefit_plan fembp
    ON fem.family_eligibility_member_id = fembp.family_eligibility_member_id
JOIN family_eligibility_member_benefit_plan_provider fembpp
    ON fembp.family_eligibility_member_benefit_plan_id
       = fembpp.family_eligibility_member_benefit_plan_id
    AND fembpp.effective_thru IS NULL
JOIN provider_identifier pid
    ON fembpp.provider_identifier_id = pid.provider_identifier_id
JOIN provider_name pn
    ON fembpp.provider_id = pn.provider_id AND pn.is_primary = 1
WHERE mn.name_last = 'Garcia-TRAIN';
-- Expected: Maria Garcia-TRAIN -> Rodriguez-TRAIN, VIC40001
```

The Group Chain (Claim to Client Group)

"Given a claim, find the employer group." This is the 6-table join used across all report SPROCs.

```
claim
  -> family_eligibility_member_benefit_plan  (ON fembp_id)
  -> family_eligibility_member               (ON family_eligibility_member_id)
  -> family_eligibility                      (ON family_eligibility_id)
  -> benefit_contract                        (ON benefit_contract_id)
  -> client_group                            (ON client_group_id)
```

```
  -> client_group_name                          (ON client_group_id, primary_name = 1)
```

```sql
-- Group chain: Claim to Employer Group (try with TRAIN-CLM-001)
SELECT c.claim_id, c.claim_ud,
       mn.name_first + ' ' + mn.name_last AS member_name,
       cgn.client_group_nm AS employer_group
FROM claim c
JOIN family_eligibility_member_benefit_plan fembp
    ON c.family_eligibility_member_benefit_plan_id
       = fembp.family_eligibility_member_benefit_plan_id
JOIN family_eligibility_member fem
    ON fembp.family_eligibility_member_id = fem.family_eligibility_member_id
JOIN member m ON fem.member_id = m.member_id
JOIN member_name mn ON m.member_id = mn.member_id AND mn.is_primary = 1
JOIN family_eligibility fe
    ON fem.family_eligibility_id = fe.family_eligibility_id
JOIN benefit_contract bc
    ON fe.benefit_contract_id = bc.benefit_contract_id
JOIN client_group cg
    ON bc.client_group_id = cg.client_group_id
JOIN client_group_name cgn
    ON cg.client_group_id = cgn.client_group_id AND cgn.primary_name = 1
WHERE c.claim_ud = 'TRAIN-CLM-001';
-- Expected: TRAIN-CLM-001, Maria Garcia-TRAIN, TRAIN - Acme Manufacturing
```

Universal Guard Clauses

These filters appear in almost every query:

Table	Filter	Purpose
`member_name`	`is_primary = 1`	One display name per member
`provider_name`	`is_primary = 1` (or ROW_NUMBER pattern)	One display name per provider
`client_group_name`	`primary_name = 1`	One display name per group
`claim_procedure`	`is_system_generated = 0 AND charge > 0`	Real billable services only
`fembpp` / `fembpuarc`	`effective_thru IS NULL`	Currently active records
`adjudication_status`	`golden_key = 'APPROVED'`	Approved claims only

Power User Tools: Navigating QC

The F8 Key — Your Rosetta Stone

When you're in any field in the QC UI, pressing F8 opens the **Control Properties** box. This shows you the exact database column name that field maps to. Here's what it looks like with the cursor in the "Last Name" field on the Member screen:

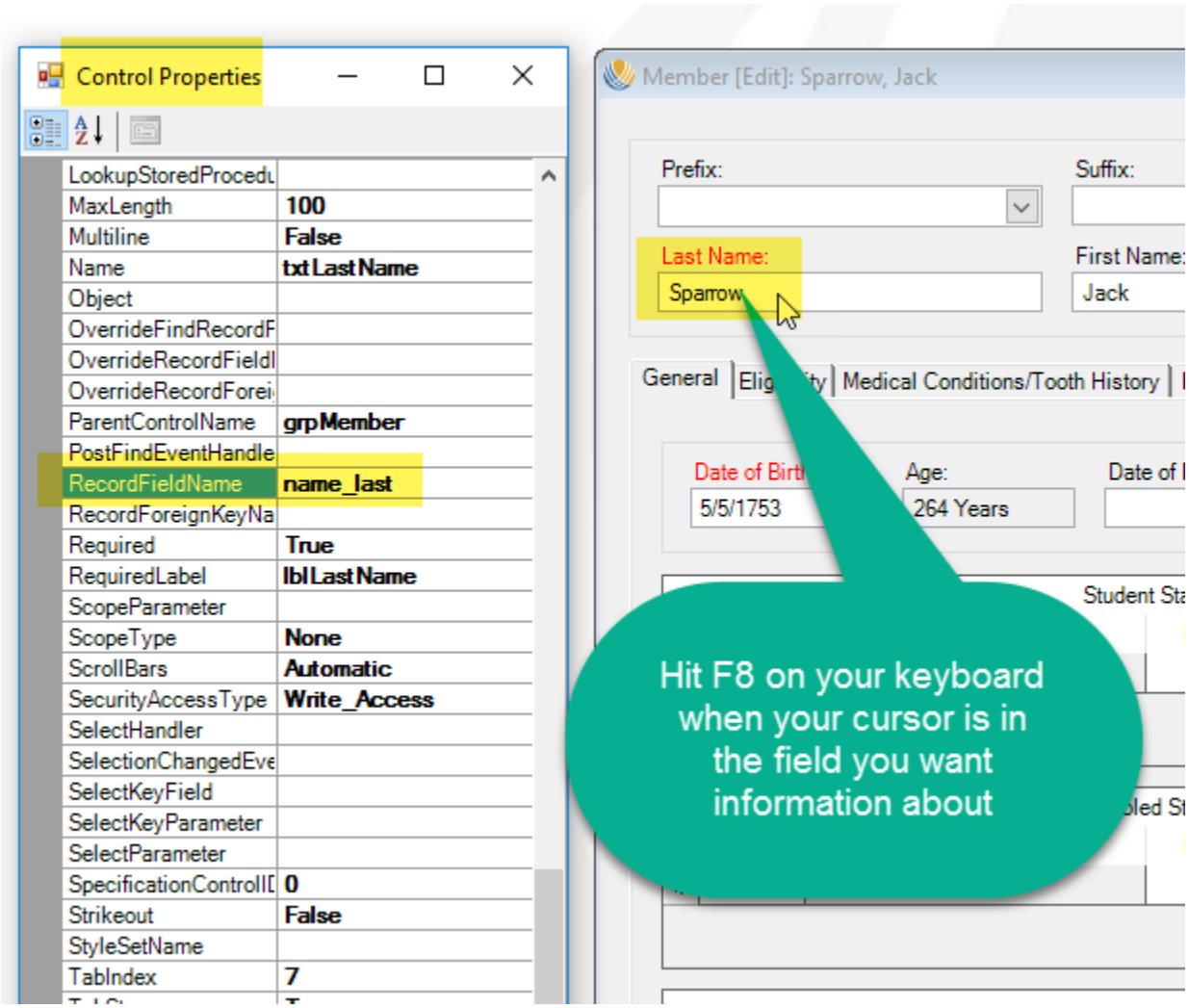

Figure 20: F8 Control Properties showing field-to-column mapping

Notice `RecordFieldName = name_last` – that tells you this UI field reads from/writes to `member_name.name_last`. This is the single most useful tool for mapping UI to database.

SQL Server Profiler — Reverse Engineering the UI

Start a SQL Profiler trace (SSMS > Tools > SQL Server Profiler), then perform an action in QC. The trace shows every stored procedure the UI calls. Here you can see QC loading a member record – it fires dozens of `plx_member_*` stored procedures:

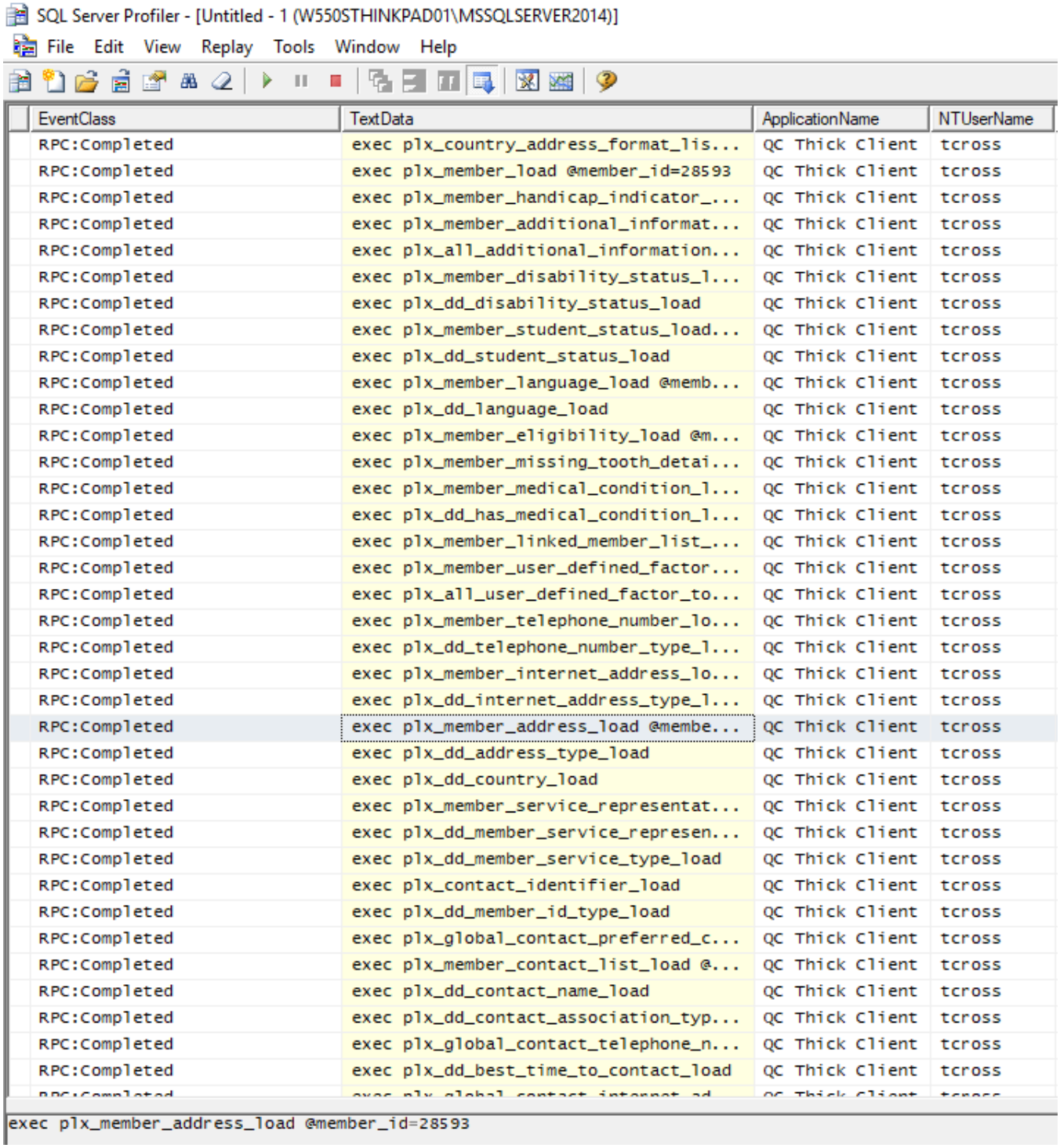

Figure 21: SQL Profiler trace showing QC stored procedures

System Table Editor — Golden vs User Data

QC has two kinds of lookup data. **Golden data** (gold shield icon) is system-protected and cannot be edited. **User data** (notepad icon) is configurable per client:

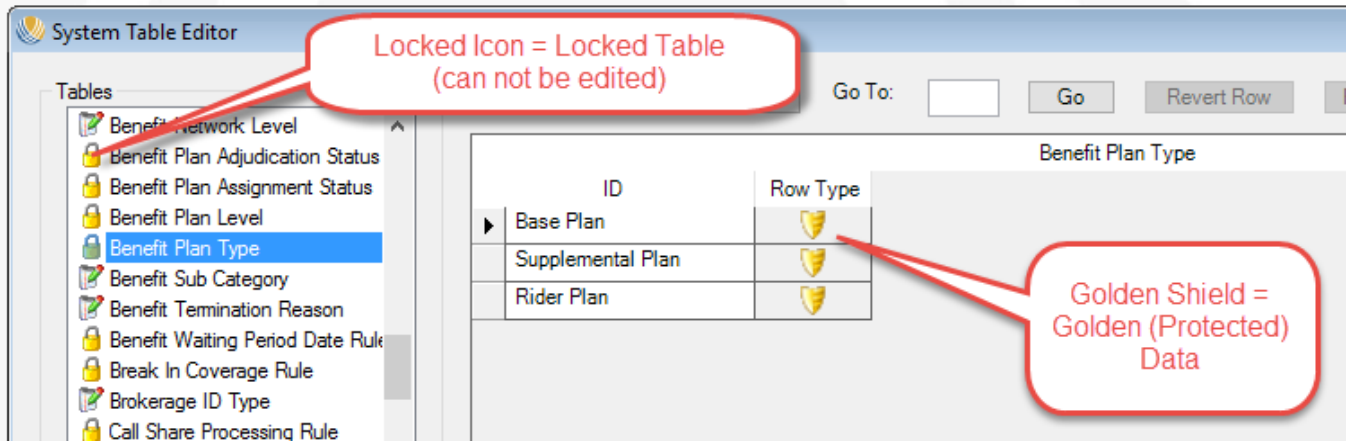

Figure 22: System Table Editor - Golden (protected) data

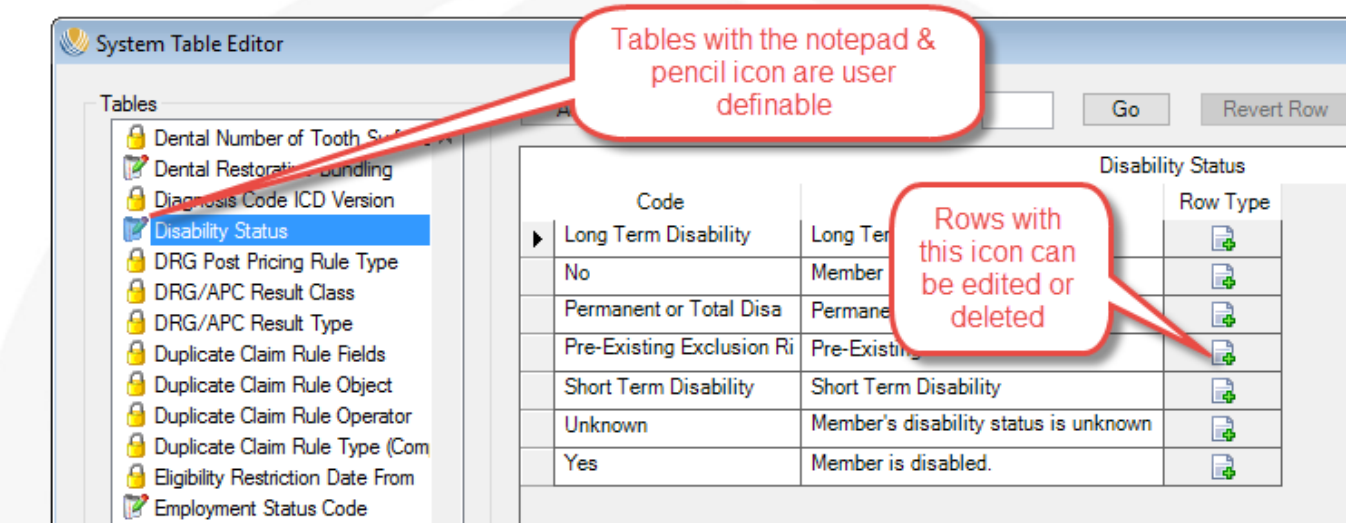

Figure 23: System Table Editor - User definable data

Glossary: Insurance Terms to qc_core Tables

Insurance Term	qc_core Entity	Plain English
TPA (Third Party Administrator)	`client`	The company administering the health plan
Employer Group	`client_group`	The company whose employees are covered
Benefit Contract	`benefit_contract`	Legal agreement between TPA and employer
Plan Year	`benefit_contract_period`	Annual coverage window
Benefit Plan	`benefit_plan`	Specific plan (e.g., “HMO Gold”)
Plan Type	`benefit_framework`	Category (HMO, PPO, POS)
Subscriber	`member` (via `family_eligibility_member`)	The employee who enrolled
Dependent	`member` (via `family_eligibility_member`)	Spouse, child covered under subscriber

Insurance Term	qc_core Entity	Plain English
Family	`family_eligibility`	Household unit for eligibility
Enrollment	`family_eligibility_member_benefit_plan`	A person enrolled in a specific plan
PCP	`provider` + `fembp_provider`	Primary Care Provider assigned to member
IPA	Derived from `provider_identifier.provider_ud` prefix	Independent Practice Association (provider network)
QCAP Code	`result_code` (where `result_code_ud LIKE 'QCAP%'`)	Tags member to their IPA
Referral	`referral`	Authorization to see specialist
Claim	`claim`	Bill submitted by provider
Claim Line	`claim_procedure`	Individual service on a claim
Adjudication	`adjudication_result_amount` + related	Process of determining payment
Copay	`adjudication_result_amount.copay_amount`	Flat fee member pays
Deductible	`adjudication_result_amount.deductible_amount`	Amount member pays before insurance kicks in
Coinsurance	`adjudication_result_amount.coinsurance_amount`	Percentage member pays after deductible
Write-off	`adjudication_result_amount.write_off_amount`	Discount from contracted rate
Net Payment	`adjudication_result_amount.net_payment`	What the plan pays the provider
EOB	Generated from A/P + claim data	Explanation of Benefits sent to member
Payment Run	`claim_payment_run`	Batch payment processing event
Capitation	`capitation_*` tables	Pre-paid monthly per-member fee to IPA
Premium	`premium` + A/R transaction tables	Monthly fee employer pays for coverage

---

This document is a companion to QC-Core-Domain-Analysis. For formal DDD concepts (aggregates, bounded contexts, integration patterns), see that reference. For the QCAP sync production script, see `clients/Verda/Verda-RT-27730/production/v4/weekly-qcap-sync.sql`.